

NUMPY ARRAY VS PYTHON LIST

```
In [3]: #LIST
a = [i for i in range(10000000)]
b = [i for i in range(10000000,20000000)]
c = []
import time
start = time.time()
for i in range(len(a)):
    c.append(a[i] + b[i])
print(time.time()-start)
```

4.6168553829193115

```
In [4]: #NUMPY
import numpy as np
a = [i for i in range(10000000)]
b = [i for i in range(10000000,20000000)]
c = []
start = time.time()
c = a+b
print(time.time()-start)
```

0.20905470848083496

```
In [7]: # memory
a = [i for i in range(10000000)]
import sys

sys.getsizeof(a)
```

Out[7]: 89095160

```
In [9]: a = np.arange(10000000,dtype = np.int8)
sys.getsizeof(a)
```

Out[9]: 10000112

```
In [10]: #BOOLEAN INDEXING
a = np.random.randint(1,100,24).reshape(6,4)
a
```

```
Out[10]: array([[82, 57, 62,  8],
                [84, 35, 33, 86],
                [36, 44, 99,  1],
                [50, 49, 45, 59],
                [34, 73,  5, 28],
                [56,  4, 73, 36]], dtype=int32)
```

```
In [11]: # find all numbers greater than 50
a[a > 50]
```

```
Out[11]: array([82, 57, 62, 84, 86, 99, 59, 73, 56, 73], dtype=int32)
```

```
In [12]: # find out even numbers
a[a % 2 ==0]
```

```
Out[12]: array([82, 62,  8, 84, 86, 36, 44, 50, 34, 28, 56,  4, 36], dtype=int32)
```

```
In [15]: # find all numbers greater than 50 and are even  
a[(a>50) & (a % 2 ==0)]
```

```
Out[15]: array([82, 62, 84, 86, 56], dtype=int32)
```

```
In [18]: #find all numbers which are not divisible by 7  
a[~(a % 7 ==0)]
```

```
Out[18]: array([82, 57, 62,  8, 33, 86, 36, 44, 99,  1, 50, 45, 59, 34, 73,  5,  
              4,              73, 36], dtype=int32)
```

Working with missing values

```
In [19]: a = np.array([1,2,3,4,5,np.nan,6,7])  
a
```

```
Out[19]: array([ 1.,  2.,  3.,  4.,  5., nan,  6.,  7.])
```

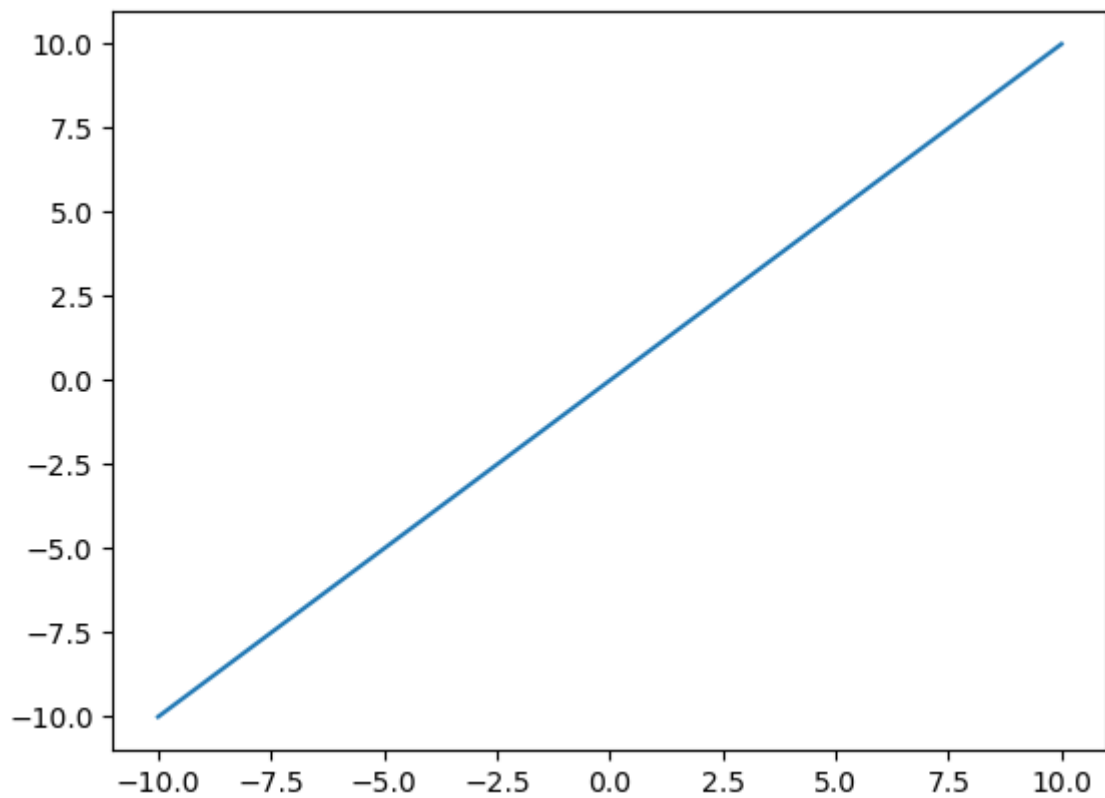
```
In [22]: a[~np.isnan(a)]
```

```
Out[22]: array([1., 2., 3., 4., 5., 6., 7.])
```

PLOTTING GRAPH

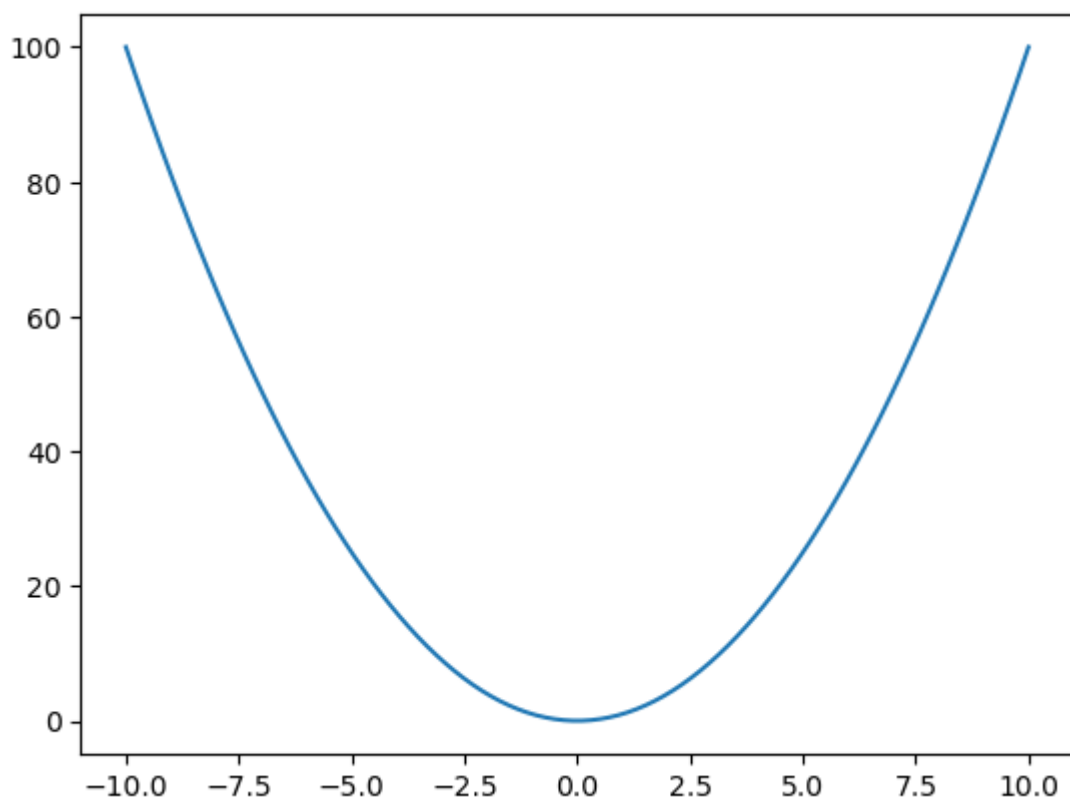
```
In [23]: #Y = X  
import matplotlib.pyplot as plt  
x = np.linspace(-10,10,100)  
y = x  
plt.plot(x,y)
```

```
Out[23]: [<matplotlib.lines.Line2D at 0x15600612e90>]
```



```
In [24]: # x^2
x = np.linspace(-10,10,100)
y = x**2
plt.plot(x,y)
```

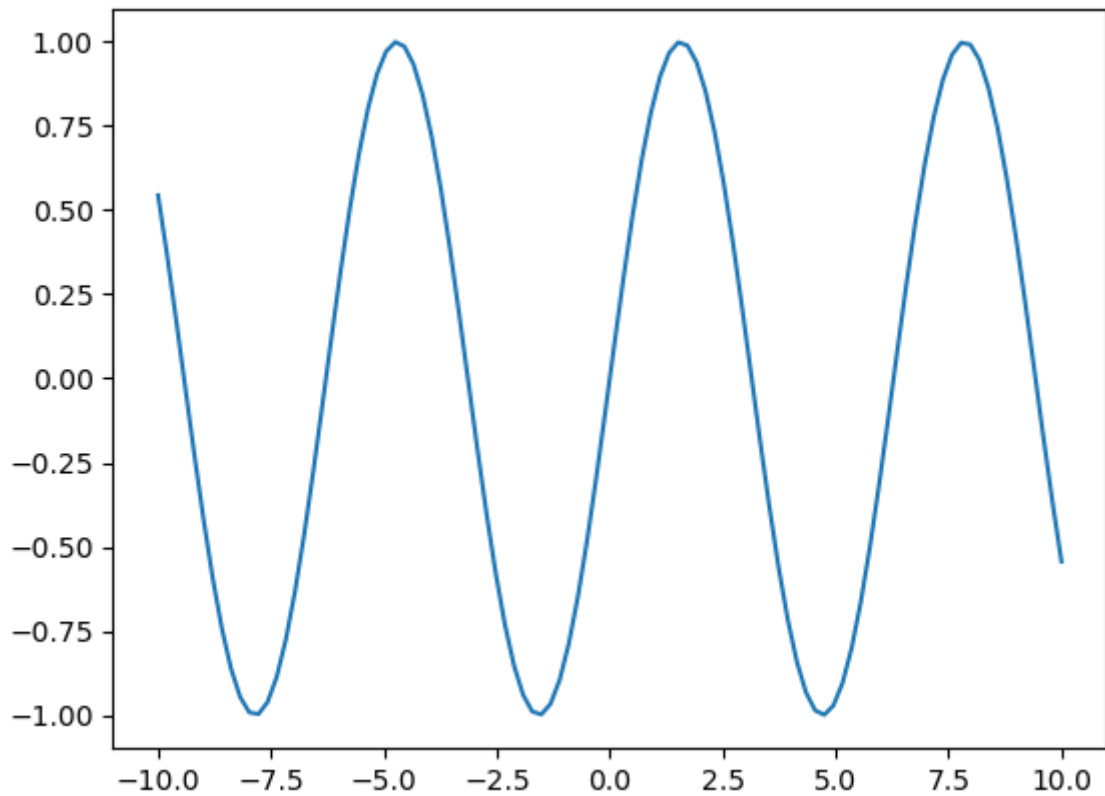
Out[24]: [



```
In [27]: # y = sinx
x = np.linspace(-10,10,100)
```

```
y = np.sin(x)
plt.plot(x,y)
```

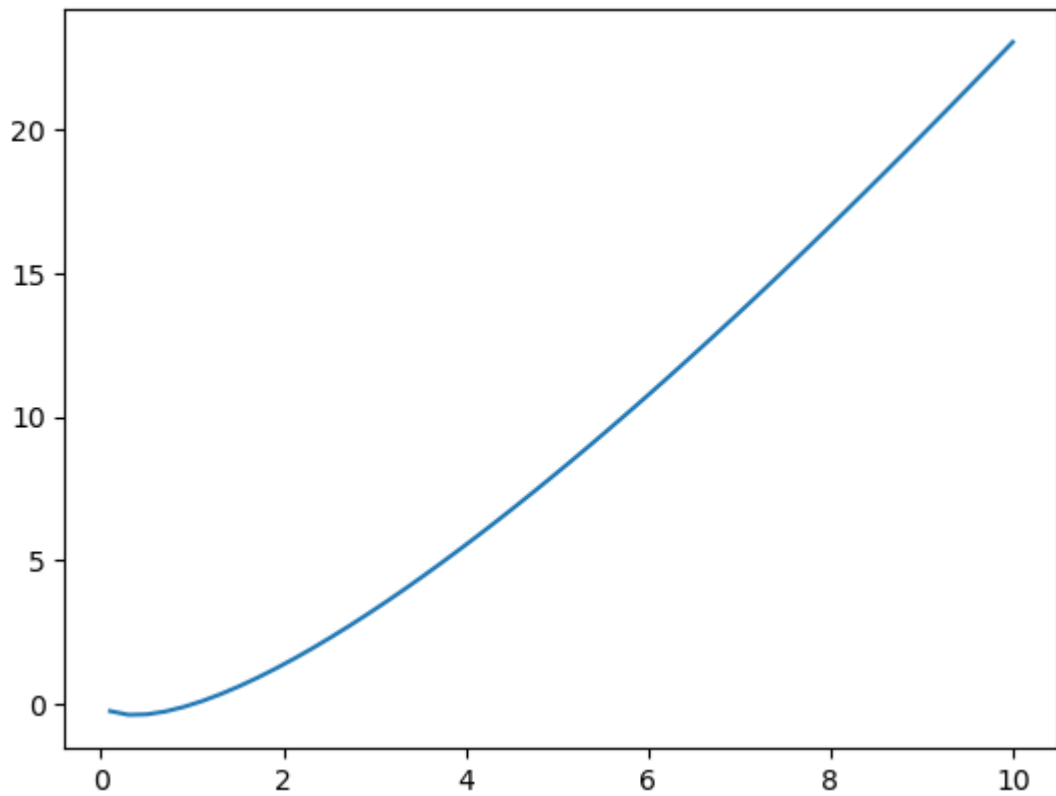
Out[27]: [



```
In [29]: # y = xlog(x)
x = np.linspace(-10,10,100)
y = x * np.log(x)
plt.plot(x,y)
```

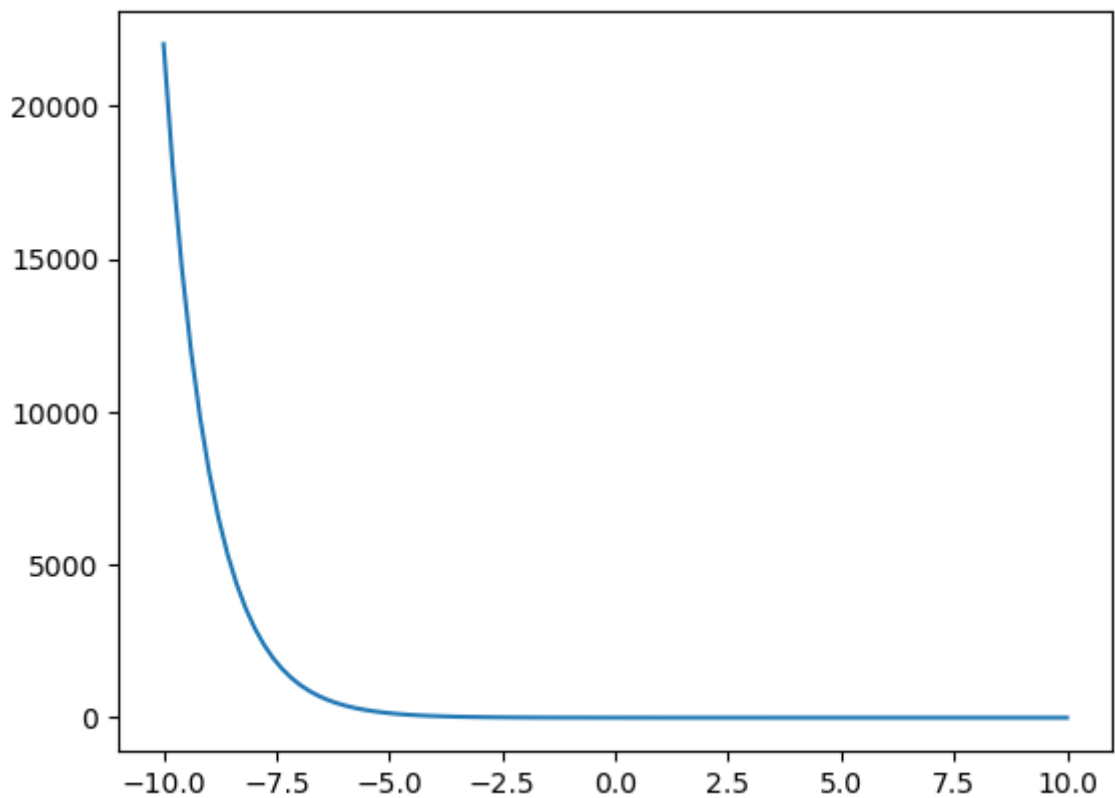
C:\Users\Admin\AppData\Local\Temp\ipykernel_7476\2649979385.py:3: RuntimeWarning: invalid value encountered in log
y = x * np.log(x)

Out[29]: [



```
In [30]: # sigmoid
x = np.linspace(-10,10,100)
y = 1/(1+np.exp(-x))
plt.plot(x,y)
```

Out[30]: [



BROADCASTING

```
In [ ]: ### Broadcasting
```

The term broadcasting describes how NumPy treats arrays **with** different shapes. The smaller array **is** "broadcast" across the larger array so that they have the same shape.

```
In [33]: # same shape  
a = np.arange(6).reshape(2,3)  
b = np.arange(6,12).reshape(2,3)  
  
print(a)  
print(b)  
  
print(a + b)
```

```
[[0 1 2]  
 [3 4 5]]  
[[ 6  7  8]  
 [ 9 10 11]]  
[[ 6  8 10]  
 [12 14 16]]
```

```
In [36]: # diff shape  
a = np.arange(6).reshape(2,3)  
b = np.arange(3).reshape(1,3)  
print(a)  
print(b)  
  
print(a + b)
```

```
[[0 1 2]  
 [3 4 5]]  
[[0 1 2]]  
[[0 2 4]  
 [3 5 7]]
```

```
In [ ]: #### Broadcasting Rules
```

****1.** Make the two arrays have the same number of dimensions.****
**
- If the numbers of dimensions of the two arrays are different, add new dimensions **with** size **1** to the head of the array **with** the smaller number of dimensions.

```
In [ ]: **2. Make each dimension of the two arrays the same size.**<br>  
- If the sizes of each dimension of the two arrays do not match, dimension mismatch will occur.  
- If there is a dimension whose size is not 1 in either of the two arrays, an error will occur.
```

```
In [39]: # More examples  
  
a = np.arange(12).reshape(4,3)  
b = np.arange(3)  
  
print(a)  
print(b)  
print(a + b)
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
[0 1 2]
[[ 0  2  4]
 [ 3  5  7]
 [ 6  8 10]
 [ 9 11 13]]
```

```
In [42]: a = np.arange(12).reshape(3,4)
        b = np.arange(3).reshape(1,3)

        print(a)
        print(b)
        print(a + b
              )
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[0 1 2]]
```

```
In [43]: a = np.arange(12).reshape(3,4)
        b = np.arange(3)

        print(a)
        print(b)

        print(a+b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[0 1 2]
```

```
-----
-
ValueError                                Traceback (most recent call last)
Cell In[43], line 7
      4 print(a)
      5 print(b)
----> 7 print(a+b)

ValueError: operands could not be broadcast together with shapes (3,4)
(3,)
```

```
In [44]: a = np.arange(3).reshape(1,3)
        b = np.arange(3).reshape(3,1)

        print(a)
        print(b)

        print(a+b)
```

```
[[0 1 2]]
[[0]
 [1]
 [2]]
[[0 1 2]
 [1 2 3]
 [2 3 4]]
```

```
In [45]: a = np.arange(3).reshape(1,3)
        b = np.arange(4).reshape(4,1)

        print(a)
        print(b)

        print(a + b)
```

```
[[0 1 2]]
[[0]
 [1]
 [2]
 [3]]
[[0 1 2]
 [1 2 3]
 [2 3 4]
 [3 4 5]]
```

```
In [46]: a = np.array([1])
        # shape -> (1,1)
        b = np.arange(4).reshape(2,2)
        # shape -> (2,2)

        print(a)
        print(b)

        print(a+b)
```

```
[1]
[[0 1]
 [2 3]]
[[1 2]
 [3 4]]
```

```
In [47]: a = np.arange(12).reshape(3,4)
        b = np.arange(12).reshape(4,3)

        print(a)
        print(b)

        print(a+b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```



```

-----
-
ValueError                                Traceback (most recent call las
t)
Cell In[47], line 7
      4 print(a)
      5 print(b)
----> 7 print(a+b)

ValueError: operands could not be broadcast together with shapes (3,4) (4,
3)

```

```
In [48]: a = np.arange(16).reshape(4,4)
        b = np.arange(4).reshape(2,2)
```

```

print(a)
print(b)

print(a+b)

```

```

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[[0 1]
 [2 3]]

```

```

-----
-
ValueError                                Traceback (most recent call las
t)
Cell In[48], line 7
      4 print(a)
      5 print(b)
----> 7 print(a+b)

ValueError: operands could not be broadcast together with shapes (4,4) (2,
2)

```

```
In [ ]:
```